



Welcome to the second part of our adventure game course in **Godot**. In this part we will focus on building all the UI elements for our game. Specifically, we will create a sophisticated inventory system that allows players to select weapons, shields, and style items, while also providing a preview of these items.

Prerequisites

Before we begin, it is essential to have a basic understanding of the following concepts in Godot:

- Basic knowledge of UIs in Godot
- Familiarity with anchors and containers
- Understanding of basic layout nodes such as color rectangles, HBox containers, and labels
- Basic knowledge of GDScript

Additionally, this course builds upon the game created in the earlier part of this series. While it is not mandatory to have watched the first part, it would be beneficial to have that foundational knowledge.

Course Overview

In this part of the course, we will cover the following topics:

1. **Creating the Inventory System:** We will use the Godot layout system to create an inventory that displays weapons, shields, and style items. These items can be equipped, and they will be visible in the game.
2. **3D Preview:** We will implement a 3D preview feature that allows players to see how their character will look with the selected items.
3. **Working with Complex Layout Projects:** We will explore a broad range of layout nodes and use anchors and containers to create the layout.
4. **Adding Interactivity with GDScript:** We will add logic via GDScript to make the inventory system interactive.

Getting Started

In the next video, we will review the basics of what we are starting with. This will help you understand the fundamental aspects of the project. If you are only interested in the menu itself, you can start here. However, watching the first part of the series will make this part much easier to follow.

If you find yourself confused at any point, you can refer back to the first part of the series and review specific sections. The entire course is designed to be modular, allowing you to focus on the parts that interest you the most.

Conclusion

This part of the course will equip you with the skills needed to create a sophisticated inventory system in Godot. By the end of this section, you will have a functional inventory with a 3D preview feature, enhancing the player's experience in your adventure game.

We look forward to seeing the amazing games you will create with these new skills!

Course Updated to Godot 4.5

We've updated the project files to Godot version 4.5 for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

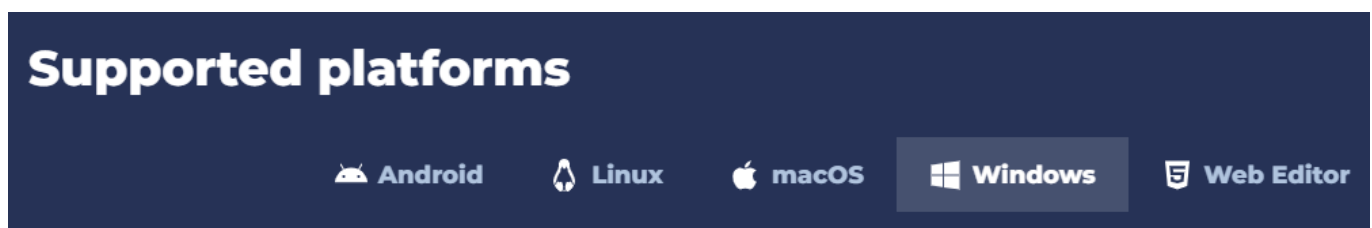
**Godot Engine – .NET****4.5**

x86_64 · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?



If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:





This course builds upon a project from Unit 1 so you can focus on the inventory and UI aspects covered in this unit. If you skipped Unit 1, you can download a project starter file from the Course Files tab located at the bottom of any lesson page or the course's home page!

Need a visual guide? Try out our helpful article on the topic: [How can I download the source code, or asset files, for my course?](#)



Welcome to this beginner-friendly guide on creating an inventory system for a 3D game using Godot. This tutorial assumes you have a basic 3D game setup, as covered in the first part of this series. If you're new to 3D game development, we recommend starting with the first part before proceeding. However, if you're only interested in the inventory system, you can follow along with this tutorial.

Prerequisites

- Basic understanding of Godot.
- A working 3D game project with a character class, player, and enemies.
- Familiarity with GDScript.

Project Overview

Before we dive into creating the inventory system, let's understand what we have so far:

- **Character Class:** The parent class for the player and enemies, handling movement and equipment logic.
- **Equipment:** The player can equip different weapons, shields, and style items.
- **Enemies:** Randomized appearances and weapons.
- **Game Features:** Character movement, defend and attack animations, weapon and shield toggling, and basic platformer mechanics.

Project Structure

The project contains the following key components:

1. **Game Scene:** Includes world environment, lighting, terrain, and characters.
2. **Character Class:** Contains collision shape, animation tree, timer, sound, and script for character logic.
3. **Player and Enemies:** Inherit from the character class, with additional logic for movement, attacks, and health.
4. **Global Scene:** Stores data for items, weapons, and shields.

Character Class Details

The character class includes the following components:

- **Animation Tree:** Handles attack, defend, and movement animations.
- **Script:** Contains variables for speed, jump logic, health, and functions for gravity, movement, and attacks.
- **Skin Variable:** Defines the character's appearance.

Player and Enemy Logic

The player and enemies have specific logic for movement and attacks:

- **Player:** Movement and jump logic based on input, attack logic based on controller or keyboard input.
- **Enemies:** Movement logic based on relative position to the player, attack logic based on a timer.

Items and Weapons

The game includes a variety of items and weapons, with data stored in a global scene:



- **Weapons:** Each weapon has a dictionary with data such as damage, thumbnail, animation, range, and audio.
- **Shields:** Similar setup to weapons, with data for defense.

Project Folders

The project folders include:

- **Audio Files:** Sound effects for the game.
- **Global Scene:** Stores item data.
- **Graphics:** Contains images and thumbnails for the inventory.
- **Scenes:** Includes game scenes and character models.
- **Shaders:** Visual effects for the game.

Getting Started with the Inventory

Now that you have a basic understanding of the project structure and components, we can start creating the inventory system. The inventory will allow the player to collect, store, and manage items and weapons.

Stay tuned for the next part of this series, where we will dive deeper into the implementation of the inventory system.



In this lesson, we will create a Heads-Up Display (HUD) for our game using Godot. The HUD will display the player's health via hearts and the equipped weapons. We will use an HFlowContainer to manage the layout of the hearts. By the end of this lesson, you will have a functional HUD that updates based on the player's health.

Creating the HUD Scene

Let's start by creating a new scene for the HUD:

1. Create a new scene with a `UserInterface` as the parent node.
2. Rename the parent node to `HUD`.
3. Save the scene inside the `scenes/ui` folder as `HUD.tscn`.

Adding a Heart Texture

To ensure our HUD is working, let's add a heart texture:

1. Add a `TextureRect` to the HUD scene.
2. Set the texture to `heart.png` located in the `res://graphics/ui` folder.
3. Add the HUD scene to the game scene to verify that the heart appears in the game.

Setting Up the HFlowContainer

Next, we will set up the `HFlowContainer` to manage the layout of the hearts:

Add an `HFlowContainer` to the HUD scene.
Set the layout mode to `Anchors` with a custom anchor preset.
Configure the anchor points and offsets to position the container in the top-left corner of the screen:

```
right: 0.25  
bottom: 0.1  
left_offset: 15  
top_offset: 15
```

Creating the Heart Scene

We will create a separate scene for the heart to manage its properties:

1. Create a new scene with a `UserInterface` as the parent node and rename it to `Heart`.
2. Add a `TextureRect` to display the heart texture (`heart.png`).
3. Save the scene as `heart.tscn` in the `scenes/ui` folder.

Configuring the Heart Scene

Let's configure the heart scene to ensure it fits within the `HFlowContainer`:

Set a custom minimum size for the heart's layout node:

```
custom_minimum_size: 16px by 16px
```

1. Set the layout preset to `Top Left`.
2. For the `TextureRect`, set the expand mode to `Fit Height` and scale to `Keep Aspect Centered`.



Adding Hearts to the HFlowContainer

Now, we will add the heart scene to the HFlowContainer and test it:

1. Add the heart.tscn scene to the HFlowContainer.
2. Duplicate the heart to ensure it arranges correctly within the container.
3. Test the game to verify that the hearts appear correctly.

Connecting Player Health to the HUD

Finally, we will write a script to connect the player's health to the HUD:

Add a script to the HUD scene that extends Control.
In the script, save the preloaded scene variable for the heart.
Create a function to set up the hearts based on the player's health. We do this simply by instantiating the hearts in the container as a child:

```
extends Control

var heart_scene = preload("res://scenes/ui/heart.tscn")

func setup(hearts: int):
    for heart in hearts:
        var heart_instance = heart_scene.instantiate()
        $Hearts.add_child(heart_instance)
```

Now, in the player script, we first need to get a reference to the HUD.

```
@onready var hud = get_tree().get_first_node_in_group("HUD")
```

Then, in the `_ready` function, we'll call the `setup` function so the HUD appears on load.

```
func _ready() -> void:
    # ... existing code ...
    hud.setup(health)
```

By following these steps, you will have a functional HUD that displays the player's health using hearts. This setup ensures that the HUD updates dynamically based on the player's health, providing a clear visual indication of the player's status during gameplay.



In this lesson, we will implement the heart logic for the player character in our game using Godot. This involves updating the hearts displayed in the HUD (Heads-Up Display) whenever the player's health changes. We will also add input controls to test the health changes easily. Let's break down the steps:

Step 1: Adding Input Controls for Testing

First, we need to add input controls to increase or decrease the player's health for testing purposes. This will be done in the `_input` function of the `player.gd` script.

```
func _input(_event: InputEvent) -> void:
    ...
    if Input.is_action_just_pressed("ui_focus_next"):
        health += 1
    if Input.is_action_just_pressed("ui_cancel"):
        health -= 1
```

Here, we use `ui_focus_next` (Tab key) to increase health and `ui_cancel` (Escape key) to decrease health. Now, if needed, we can press these keys to ensure our UI works (without needing to initiate any sort of combat).

Step 2: Updating the Health Logic

Next, we need to ensure that the health variable is updated correctly and that the HUD reflects these changes. This involves modifying the health setter in the `character.gd` script.

```
var health: int = 3:
    set(value):
        if "hud" in self:
            get_tree().get_first_node_in_group("HUD").set_health(value)
            if value <= 0:
                get_tree().quit()
            health = value
```

In this setter, we check if the HUD is in the scene. If it is, we call the `set_health` function on the HUD node and pass the new health value. If the health is less than or equal to zero, we quit the game.

Step 3: Implementing the `set_health` Function in the HUD

Now, we need to implement the `set_health` function in the `hud.gd` script. This function will update the hearts displayed in the HUD.

```
func set_health(health: int):
    for child in $Hearts.get_children():
        child.queue_free()
    setup(health)
```



In this function, we first remove all existing heart nodes from the Hearts container. Then, we call the setup function to add the correct number of hearts based on the new health value.

Step 4: Testing the Implementation

Finally, we test our implementation by running the game and using the Tab and Escape keys to increase and decrease the player's health. We should see the hearts in the HUD update accordingly.

By following these steps, we have successfully implemented the heart logic for the player character in our game. This ensures that the HUD accurately reflects the player's health, providing valuable feedback during gameplay.



In this lesson, we will focus on implementing a pause and resume functionality for our game using the Godot engine. This feature will allow players to pause the game, view their equipped weapons, shields, and style items, and then resume gameplay. Godot provides a straightforward way to achieve this through its built-in functionality.

Pausing the Game

To pause the game, we need to stop all process functions. This can be achieved by setting the paused property of the scene tree to true. Here's how you can do it:

```
get_tree().paused = true
```

This line of code will pause all processes in the game, effectively freezing the game state.

Implementing Pause Logic in Player Script

We will start by modifying the player.gd script to handle the pause input. Specifically, we will work within the `_input` function. Follow these steps:

1. Open the player.gd script.
2. Locate the `_input` function.
3. Add the following code to handle the pause input:

```
if Input.is_action_just_pressed("menu"):
    get_tree().paused = true
```

The menu action is typically mapped to the Start button on a controller or the M key on the keyboard. You can verify this mapping in the project settings under the input map.

Creating a Pause Function in Global Script

To make the pause functionality more modular, we will create a pause function in the global.gd script. This function will handle pausing and resuming the game. Here's how to do it:

1. Open the global.gd script.
2. Add the following function:

```
func pause(value: bool = true):
    get_tree().paused = value
```

This function takes a boolean parameter value that defaults to true. When called, it sets the paused property of the scene tree to the value of value.

Updating Player Script to Use Global Pause Function

Now, we will update the player.gd script to use the newly created pause function in the global.gd script. Follow these steps:

1. Open the player.gd script.



2. Modify the `_input` function to call the global pause function:

```
if Input.is_action_just_pressed("menu"):
    Global.pause()
```

This change ensures that the pause functionality is centralized in the `global.gd` script, making it easier to manage and extend.

Creating the Inventory Scene

Next, we will create a new scene for the inventory user interface. This scene will be displayed when the game is paused. Follow these steps:

1. Create a new scene of type User Interface and name it `inventory.tscn`.
2. Save the scene in the `scenes/ui` directory.
3. Add a script to the inventory scene and name it `inventory.gd`.

In the `inventory.gd` script, we will handle the input to resume the game. Add the following code:

```
func _input(event):
    if Input.is_action_just_pressed("menu"):
        Global.pause(false)
```

This code checks for the menu action and calls the global pause function with `false` to resume the game.

Adding the Inventory to the Game Scene

To display the inventory when the game is paused, we need to add the inventory scene to the game scene. Follow these steps:

1. Open the game scene.
2. Add the `inventory.tscn` scene as a child node.

To ensure the inventory is hidden by default, you can click the eye icon next to the inventory node in the scene tree.

Assigning the Inventory to a Group

To manage the visibility of the inventory, we will assign it to a group. Follow these steps:

1. Select the inventory node.
2. Go to the Groups tab in the inspector.
3. Add the inventory node to a new group named `Inventory`.

In the `global.gd` script, we will use this group to hide and show the inventory. Add the following code to the `global.gd` script:

```
@onready var inventory = get_tree().get_first_node_in_group("Inventory")

func pause(value: bool = true):
```



```
get_tree().paused = value
if value:
    inventory.show()
else:
    inventory.hide()
```

This code ensures that the inventory is shown when the game is paused and hidden when the game is resumed.

Handling Paused Input Processing

To ensure that the input function in the inventory scene is only processed when the game is paused, we need to change the process mode of the inventory scene. Follow these steps:

1. Select the root node of the inventory scene.
2. In the inspector, find the Process property.
3. Change the Mode property to When Paused.

This ensures that the input function in the inventory scene is only called when the game is paused.

Adding a Timer to Prevent Immediate Resume

To prevent the game from resuming and pausing within the same frame, we will add a timer to the global.gd script. Follow these steps:

1. Open the global.gd script.
2. Add a timer node and name it PauseTimer.
3. Modify the pause function to use the timer:

```
func pause(value: bool = true):
    if $PauseTimer.time_left == 0.0:
        get_tree().paused = value
        $PauseTimer.start()
    ...
```

Set the wait time of the PauseTimer to 0.3 seconds and ensure it is set to one-shot mode.

Ensuring Global Scene Processing

To ensure that the global scene is always processed, even when the game is paused, change the process mode of the global scene to Always.

With these steps, you have implemented a basic pause and resume functionality in your game. Players can now pause the game, view their inventory, and resume gameplay seamlessly.



Welcome to this beginner-friendly guide on creating a static inventory system in Godot. This tutorial will walk you through the process of designing an inventory interface using various layout nodes. By the end of this article, you will have a clear understanding of how to structure your inventory system and style it to look appealing.

Setting Up the Inventory Layout

To begin, ensure you are inside the Godot editor and have your inventory scene open. The first step is to create the basic structure of the inventory interface.

Creating the HBox Container

1. Delete the existing label if there is one.
2. Create an HBoxContainer that will fill the entire available space (i.e. Full Rect)

Adding Panel Containers

Next, we will add two main panels within the HBoxContainer.

1. Add a PanelContainer for the items. Ensure it expands to take up the available space. Name this one "ItemPanelContainer".
2. Duplicate the PanelContainer to create another one for the player. Name this one "PlayerPanelContainer".

Adjusting Panel Sizes

To allocate more space to the items panel, follow these steps:

1. Set the Stretch Ratio of the ItemPanelContainer to 2. This will make it take up two-thirds of the space.
2. Add a Control node between the two PanelContainers to act as a spacer. Set its custom minimum size for the x-axis to 50 pixels.

Styling the Panel Containers

To style the panel containers, follow these steps:

1. For the ItemPanelContainer, override the theme styles and add a StyleBoxFlat.
2. Set the color to a dark gray (#141414) and the corner radius to 20 pixels for each corner.
3. Repeat the same styling for the PlayerPanelContainer.

Adding a Border and Text to the Item Panel

To add a border and text to the item panel, follow these steps:

1. Add a MarginContainer to the ItemPanelContainer.
2. Add a Panel to the MarginContainer and override its theme styles with a StyleBoxFlat.
3. Set the border color to a greenish color (#59e800) and the border width to 5 pixels. Add a corner radius of 10 pixels.
4. Add a Control node to the item container and then add a Label as its child. Set the text to "Weapons" and center the alignment.
5. Override the theme colors for the font and set it to the same color as the border. Add a StyleBoxFlat for the background of the text with the same color.

Positioning the Text



To position the text appropriately, follow these steps:

1. Change the layout mode to anchors and set custom values. Adjust the left side offset to 54 pixels.
2. Expand the margins of the style box for the text to 10 pixels for the left and right sides.
3. Set a font for the text from the UI folder and adjust the font size to 40.

Adding the Player Preview

To add a player preview, follow these steps:

1. Copy the margin container from the item panel and paste it into the player panel.
2. Add a SubViewportContainer to the player panel.
3. Create a new scene with a SubViewport as the root node and rename it to PlayerViewport.
4. Add the 3D scene of the player (e.g., `rogue.glb`) and remove unnecessary nodes, leaving only the meshes, animation player, and skeleton.
5. Add an idle animation to the player and save the scene.
6. Add the `PlayerViewport.tscn` to the SubViewportContainer in the inventory scene.
7. Add a Camera3D to the PlayerViewport and point it at the player.
8. Adjust the SubViewportContainer to stretch and ensure the player viewport has a transparent background.

Fixing the Debugger Issue

If you encounter a debugger issue, add an underscore to the input event for the inventory (e.g., `_input_event`).

Congratulations! You have now created a basic static inventory system in Godot. This layout can be further customized and expanded based on your game's requirements. Happy coding!



In this lesson, we will focus on improving the player preview in our game. Currently, the player preview has some issues that need to be addressed. Specifically, we will work on two major tasks:

Major Tasks

- Adding a light and a world environment to the player to enhance the visuals.
- Fixing smaller issues such as animation and player position.

1. Adding Light and World Environment

First, let's address the lighting issue. The player is not visible in the preview because the lighting is not set up correctly. We need to add a sun and an environment to the scene.

1. Open the PlayerViewport scene in Godot.
2. Add a new DirectionalLight3D node to simulate the sun.
3. Add a new WorldEnvironment node to set up the environment.

2. Fixing Player Position and Animation

Next, we need to adjust the player's position and ensure the idle animation is playing correctly.

1. Select the camera in the PlayerViewport scene.
2. Set the Y transform to 1.3 and the Z transform to 2.9 to move the camera further away and up slightly.
3. Ensure the idle animation is set to loop and autoplay in the AnimationPlayer node.

3. Enabling Player Rotation in Inventory

We want the player preview to rotate when viewed in the inventory. To achieve this, we need to assign the player to a group and write a script to handle the rotation.

1. Assign the rogue character model to a group called "PlayerPreview".
2. In the inventory script, get the player preview node and implement the rotation logic.

```
extends Control
```

```
@onready var player_preview = get_tree().get_first_node_in_group("PlayerPreview")
@export var pan_speed: float = 3
```

```
func _process(delta: float) -> void:
    var pan_dir = Input.get_axis("pan_left", "pan_right")
    player_preview.rotate_y(pan_dir * delta * pan_speed)
```

This script pans a player preview horizontally based on user input. It uses the pan_left and pan_right input axes to determine the direction and speed of the pan, and multiplies this by a adjustable pan_speed value and the time elapsed since the last frame to create a smooth effect. The result is a seamless horizontal rotation of the player preview.

4. Adding Bone Attachments

Finally, we will add bone attachments to the player's skeleton to equip items like weapons and hats. This will allow us to attach scenes to the player for testing purposes.



1. Add 3 BoneAttachment3D nodes to the skeleton and select the appropriate bones (e.g., right hand, left hand, head).
2. Rename the bone attachments accordingly (e.g., RightHand, LeftHand, Head).
3. Attach scenes to the bone attachments for testing.

Conclusion

By following these steps, you should be able to improve the player preview significantly. The player should now be visible with proper lighting, the idle animation should be playing correctly, and the player preview should rotate in the inventory. Additionally, you can attach items to the player's bone attachments for testing purposes.

Feel free to customize the camera and other settings to achieve the desired look for your game. In the next lesson, we will continue to enhance the inventory.



In this lesson, we will create inventory items and display them using a grid container in Godot. This process involves two main steps: creating an inventory item scene and using a grid container to display these items. Let's break down the steps:

Step 1: Create an Inventory Item Scene

First, we need to create a new user interface scene for our inventory item. This scene should be a button so that it can be pressed. Here's how to do it:

1. Create a new user interface scene.
2. Change the root node to a Button instead of a Control node.
3. Rename the button to Item.
4. Save the scene as item.tscn inside the UI folder.

Step 2: Set Up the Grid Container

Next, we need to set up a grid container to display our inventory items. Follow these steps:

1. Open the inventory scene.
2. Add a MarginContainer to the ItemPanelContainer.
3. Add a GridContainer as a child of the MarginContainer.
4. Add the newly created Item scene to the GridContainer as a child.

Step 3: Configure the Grid Container

To make the items visible and properly spaced, we need to configure the grid container:

1. Enable the Expand property of the Item to make it fill the available space.
2. Add four columns to the GridContainer to organize the items.
3. Add multiple items (e.g., 11 items) to the GridContainer.
4. Add theme overwrite constants to the MarginContainer to set margins to 30px each.

Step 4: Style the Button

To style the button, we need to remove any text and add a theme overwrite for the button's normal state:

1. Remove any text from the button.
2. Add a theme overwrite for the button's normal state to use a style box flat:

Step 5: Add a Texture Rectangle

To display the weapon preview, add a TextureRect to the button:

1. Add a TextureRect as a child of the button.
2. Set the TextureRect to span the entire available space.
3. Set the texture to display a weapon thumbnail (e.g., dagger) to test it out:

Step 6: Dynamically Create Inventory Items

To dynamically create inventory items when the menu is opened, we need to add a function to our script:

1. Create a function called `create_inventory_items`. This will need to take in our grid container, the equipment list, and an index number.



2. Iterate through the `equipment_list` and print out the data for each item. Further below, we'll add more. For now, this is good for a first step.

```
func create_inventory_items(container: GridContainer, equipment_list: Array, index: int):  
    for i in equipment_list.size():  
        var equipment_item = equipment_list[i]  
        print(equipment_item)
```

Step 7: Call the Function

Next, we need to call the `create_inventory_items` function when the menu is revealed:

1. Create a new reveal function to call `create_inventory_items`.
2. Pass the necessary arguments: `grid_container`, `player.weapons`, and `player.weapon_index`. In order to make things easier, we'll set up a variable to store a reference to our player for easy reference.

```
@onready var player = get_tree().get_first_node_in_group("Player")
```

```
func reveal():  
    show()  
    create_inventory_items($HBoxContainer/ItemPanelContainer/MarginContainer2/GridContainer, player.weapons, player.weapon_index)
```

Likewise, we need to call the reveal function now. To do so, update the **pause function** in the Globals script to use the reveal function instead.

```
func pause(value: bool = true):  
    if $PauseTimer.time_left == 0.0:  
        get_tree().paused = value  
        $PauseTimer.start()  
    if value:  
        inventory.reveal()  
    else:  
        inventory.hide()
```

Now if we run everything, we should see our item data printed in the console.

Step 8. Instantiating Items

Now that we know our initial functions work, let's implement instantiating items.

First, in our **Inventory script**, we need to get a reference to our item scene.

```
@onready var item_scene = preload("res://scenes/ui/item.tscn")
```



On our Item, we need to be able to update the TextureRect to display our actual item data. This data is already stored in our global dictionary. As such, we just need to make a simple function that displays that data from our dictionary.

Create a new script on the Item called **item.gd** and save it. Then, set up a **setup** function that changes the TextureRect node.

```
extends Button
```

```
func setup(data):  
    $TextureRect.texture = data['thumbnail']
```

After doing this, select the Item and set its minimum size to 100px by 100px. Otherwise, the texturerect won't display.

We can now update our **create_inventory_items function**. Instead of just printing our data, we'll instead instantiate an instance of our item scene as a child. We'll also call the setup function from above to display an image.

```
func create_inventory_items(container: GridContainer, equipment_list: Array, index: int):  
    for i in equipment_list.size():  
        var equipment_item = equipment_list[i]  
        var item_instance = item_scene.instantiate()  
        item_instance.setup(equipment_item)  
        container.add_child(item_instance)
```

Now if we test our project, we'll see our weapons appear in the inventory.

By following these steps, you will have successfully created and displayed inventory items in a grid container in Godot. This setup allows for dynamic item creation and styling, making your inventory system more flexible and visually appealing.



In this lesson, we will refine the item buttons in our inventory system to make them more functional and visually appealing. We'll focus on three main tasks:

1. Scaling the Item Buttons

First, we need to ensure that the item buttons scale properly with the window size. We don't want them to have a constant size; instead, they should expand to fill the available space. Here's how we can achieve this:

- Remove the custom minimum size from the item scene. This is crucial because a fixed size would break the menu layout.
- Set the size flags to expand and fill horizontally. This can be done in the `_ready` function of the item scene:

```
func _ready() -> void:
    size_flags_horizontal = Control.SIZE_EXPAND | Control.SIZE_FILL
```

- To ensure the button expands to fill the available space, we use the following constants:

```
size_flags_horizontal = Control.SIZE_EXPAND | Control.SIZE_FILL
```

- For the vertical space, we want the button to be as tall as it is wide. We can achieve this by updating the custom minimum size whenever the button is resized. This can be done by connecting the `_on_resized` signal and function:

```
func _on_resized() -> void:
    custom_minimum_size.y = get_rect().size.x
```

2. Highlighting the Focused Item

Next, we want to highlight the item that the player is currently focusing on. Godot automatically focuses a button when it is clicked, and we can use this to style the focused button. Add a style box flat for the focused state of the button. This can be done in the Godot editor by adding a new style box flat and configuring it as follows:

- Draw only the border, not the center.
- Set the border color to a light color, such as white.
- Set the border width to 5 pixels.
- Set the corner radius to 5 pixels.

This will give the focused button a nice, styled border that indicates it is selected.

To test this, we can update our **reveal** function in our **Inventory script** to call the **grab_focus()**.

```
func reveal():
    ...
    $HBoxContainer/ItemPanelContainer/MarginContainer2/GridContainer.get_child(0).grab_focus()
```

3. Displaying the Equipped Item

Finally, we want to display which item is currently equipped. We can do this by adding a panel node



behind the texture rectangle of the item button and showing or hiding it based on whether the item is equipped.

First, add a panel node to the item scene and name it `EquippedPanel`. This panel should cover the entire available space. In the theme override styles, add a style box flat for the equipped panel. Configure it as follows:

- Draw only the border, not the center.
- Set the border color to a yellowish color.
- Set the border width to 6 pixels.
- Set the corner radius to 8 pixels (later refined to 5 pixels to match other panels).

Next, Add a function to the item scene to highlight the equipped item. In this case, we'll just toggle our new panel on and off as needed:

```
func highlight(value: bool):
    if value:
        $EquippedPanel.show()
    else:
        $EquippedPanel.hide()
```

To use this function, we'll need to update our **inventory** script to call it when we create the items. This way our currently equipped item will highlight itself.

```
func create_inventory_items(container: GridContainer, equipment_list: Array, index: int):
    for i in equipment_list.size():
        var equipment_item = equipment_list[i]
        var item_instance = item_scene.instantiate()
        item_instance.setup(equipment_item)
        container.add_child(item_instance)
        item_instance.highlight(i == index)
```

By following these steps, we can create a more functional and visually appealing inventory system. The item buttons will scale properly with the window size, highlight the focused item, and highlight the equipped item.



It's time to work on our inventory implementation for our other equippables. To reinforce your learning, try to complete the following exercise:

1. Create a tab container with three grid containers for weapons, shields, and style items.
2. Populate each grid container with the appropriate items.
3. Implement the functionality to toggle between the tabs, displaying the different item categories.

Introduction to Tab Containers

A tab container is a useful UI element that allows you to switch between different views or panels within the same space. In our case, we will use it to toggle between weapons, shields, and style items in our inventory.

Steps to Expand the Inventory

1. **Create a Tab Container:** Start by creating a tab container that will hold three grid containers, one for each item category.
2. **Replace the Current Grid Container:** Remove the existing grid container and replace it with the newly created tab container.
3. **Add Items to Appropriate Containers:**
 - The first grid container should hold weapons.
 - The second grid container should hold shields.
 - The third grid container should hold style items.
4. **Toggle Between Tabs:** Implement functionality to switch between the different tabs of the tab container, allowing you to view different grid containers with their respective items.

Focusing the First Button

To ensure a smooth user experience, make sure that the first button is always focused when changing the current tab inside the tab container. This step is crucial for maintaining intuitive navigation within your inventory system.

In the next section, we'll go over the solution step-by-step! Good luck!



In this lesson, we will expand the inventory system and ensure that the game does not crash when creating the menu. We will also implement functionality to switch between different inventory tabs using shoulder buttons on a controller or specific keys on the keyboard. Additionally, we will update the title to reflect the currently selected tab. Let's break down the steps:

Updating the Inventory Structure

First, we need to modify the structure of our inventory. Currently, we have a single grid container inside a margin container. We will replace this with a tab container that holds three grid containers, one for each category: weapons, shields, and style items.

1. Delete the existing grid container.
2. Add a TabContainer.
3. Inside the TabContainer, add three GridContainer nodes.
4. Rename the grid containers to Weapons, Shields, and Style for clarity.
5. Set the tabs_visible property of the TabContainer to false since we only care about the items.

Working with Grid Containers in Code

Next, we need to work with these grid containers in our script. To make the code more manageable, we will store references to these grid containers in variables.

```
@onready var weapon_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Weapons
@onready var shield_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Shields
@onready var style_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Style
@onready var tab_container = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer
```

Creating Inventory Items for All Grids

Since we replaced the grid container, we need to update the code to use the new weapon_grid instead of the old container to avoid crashes. We will also duplicate our function calls to set up our shields and styles as well.

```
func reveal():
    show()
    create_inventory_items(weapon_grid, player.weapons, player.weapon_index)
    create_inventory_items(shield_grid, player.shields, player.shield_index)
    create_inventory_items(style_grid, player.styles, player.style_index)
    weapon_grid().get_child(0).grab_focus()
```

Switching Between Tabs

To switch between the different containers, we will use the shoulder buttons on a controller or specific keys on the keyboard. We will map these actions in the input map.

- Use switch_weapon for the top shoulder button or the 'W' key to go forward.
- Use switch_shield for the bottom shoulder button or the 'Q' key to go backward.



We will store the input value in a variable called `menu_dir` and use it to change the current tab of the `TabContainer`. Notice the use of “posmod” here which ensures we can’t get any out of index errors and everything looks properly.

```
func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("menu"):
        Global.pause(false)
        var menu_dir = int(Input.get_axis("switch_shield", "switch_weapon"))
        if menu_dir:
            tab_container.current_tab = posmod(tab_container.current_tab + menu_dir, tab_container.get_child_count())
```

Update the Focus

When we change tabs, we’ll also need to update the focus. To do so, let’s create a **focus** function which grabs the focus. We’ll add a timer and an error check to the function to prevent errors.

```
func focus():
    await get_tree().create_timer(0.01).timeout
    if tab_container.get_child(tab_container.current_tab).get_child_count():
        tab_container.get_child(tab_container.current_tab).get_child(0).grab_focus()
```

With this function added, we can update the reveal function to use it.

```
func reveal():
    show()
    create_inventory_items(weapon_grid, player.weapons, player.weapon_index)
    create_inventory_items(shield_grid, player.shields, player.shield_index)
    create_inventory_items(style_grid, player.styles, player.style_index)
    focus()
```

We’ll then add it to our tab switching function as well.

```
func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("menu"):
        Global.pause(false)
        var menu_dir = int(Input.get_axis("switch_shield", "switch_weapon"))
        if menu_dir:
            tab_container.current_tab = posmod(tab_container.current_tab + menu_dir, tab_container.get_child_count())
            focus()
```

Updating the Title

Finally, we will update the title to reflect the currently selected tab. We will do this by setting the text of the label to the name of the currently selected grid container. We can do this after we call our **focus()** function in **_input**.



```
$HBoxContainer/ItemPanelContainer/Control/Label.text = tab_container.get_child(tab_container.current_tab).name
```

By following these steps, we have expanded the inventory system, ensured the game does not crash, implemented tab switching, and updated the title to reflect the current tab. This completes the basics of the menu functionality.



In this lesson, we will make the inventory functional by adding logic to handle item selection and equipping. We will update both the preview player and the actual player character when an inventory item is selected. Let's break down the steps required to achieve this:

Step 1: Checking for Button Press

First, we need to check if an inventory button is pressed. Since each item in the inventory is fundamentally a button, we can use the pressed signal to detect when an item is selected. Connect the signal to the **Item** script.

```
func _on_pressed() -> void:
    pass
```

Step 2: Storing Equipment Data

When setting up the item, we need to store some data. This data will be used to update the player and preview. We will create a variable called `equipment_data` to store this information.

```
var equipment_data

func setup(data):
    equipment_data = data
```

To ensure that our setup is working correctly, we will print the `equipment_data` when a button is pressed. This will help us verify that the data is being stored and retrieved correctly.

```
func _on_pressed():
    print(equipment_data)
```

Step 3: Updating the Real Player

To update the real player, we need to call the `equip` function from the `Character` class. This function requires two parameters: the equipment data and the slot where the item should be equipped. The slot is determined by the type of item (weapon, shield, or style). Since we know this data from the dictionary, we can use this to easily process which slot is required.

```
func _on_pressed():
    var slot = {
        'weapon': 'RightHand',
        'shield': 'LeftHand',
        'style': 'Head'
    }[equipment_data['type']]
    player.equip(equipment_data, player.get_node('PlayerSkin/Rogue/Rig/Skeleton3D/' + slot))
```

Step 4: Equipment Logic for the Player Preview

As our Player Preview is not inheriting from our **Character class**, we'll need to create separate



equipment logic. Create a new script on the Rogue character in the player preview scene called **Rogue.gd**.

This file will need two functions: one to equip items and one to unequip them. Let's work with equipping first. In this function, we need to once again use our equipment data to determine which slot we need to update. We then loop through all the children and clear them to prevent any equipment overlap. Finally, we can then instantiate the item and add it as a child to the model.

```
extends Node3D
```

```
func add_equipment(data):
    var slot = {
        'weapon': $Rig/Skeleton3D/RightHand,
        'shield': $Rig/Skeleton3D/LeftHand,
        'style': $Rig/Skeleton3D/Head
    }[data['type']]
    for child in slot.get_children():
        child.queue_free()
    var item_instance = data['scene'].instantiate()
    slot.add_child(item_instance)
```

Step 6: Updating the Preview Player

Back in **Item.gd**, to update the preview player, we will call `add_equipment` on the `PlayerPreview` node.

```
func _on_pressed():
    ...
    player_preview.add_equipment(equipment_data)
```

Step 7: Highlighting the Selected Item

Finally, we want to highlight the selected item in the inventory to indicate that it is equipped. We will call the `highlight` function on the item button and pass `true` as the parameter. Additionally, we will ensure that only one item is highlighted at a time by iterating through all items and setting their `highlight` to `false` before highlighting the selected item.

```
func _on_pressed():
    ...
    for item in get_parent().get_children():
        item.highlight(false)
    highlight(true)
```

Complete `_on_pressed` Function

Here is the complete `_on_pressed` function that incorporates all the steps mentioned above:

```
func _on_pressed():
    # Update the real player
    var slot = {
```



```
        'weapon': 'RightHand',
        'shield': 'LeftHand',
        'style': 'Head'
    }[equipment_data['type']]
    player.equip(equipment_data, player.get_node('PlayerSkin/Rogue/Rig/Skeleton3D/' +
slot))

    # Update the preview
    player_preview.add_equipment(equipment_data)

    # Highlight the selected item
    for item in get_parent().get_children():
        item.highlight(false)
    highlight(true)
```

By following these steps, we have successfully implemented the logic to make the inventory functional. The player can now select items from the inventory, and both the preview player and the actual player character will be updated accordingly. Additionally, the selected item will be highlighted in the inventory to provide visual feedback to the player.

In this lesson, we will complete our project by addressing a few minor tasks. Here's what we will accomplish:

Tasks to Complete:

- Add specific colors for different item types.
- Fix the preview character on startup.
- Allow the player to unequip hats from the menu.

Adding Specific Colors for Item Types

First, let's add specific colors for different item types. We will define these colors in the `inventory.gd` script. We want to have different border colors for weapons, shields, and style items.

Inside `inventory.gd`, we will define an array of colors:

```
const border_colors = [Color.LIME_GREEN, Color.FIREBRICK, Color.GOLDENROD]
```

Next, we need to apply these colors based on the current tab. We will modify the input logic to change the border color when switching tabs. Here's how to do it:

1. Get the current color from the `border_colors` array using the current tab index.
2. Update the theme style box of the border panel to use the new color.

Here is the code snippet to achieve this:

```
var color = border_colors[tab_container.current_tab]
$HBoxContainer/ItemPanelContainer/MarginContainer/Panel.get_theme_stylebox("panel").border_color = color
$HBoxContainer/ItemPanelContainer/Control/Label.add_theme_color_override("font_color", color)
```

Fixing the Preview Character on Startup

Next, we need to ensure that the preview character displays the correct equipment when the menu is opened. Currently, the preview character holds the wrong weapons. We will fix this by updating the `create_inventory_items` function in `inventory.gd`.

Here is the updated code snippet:

```
func create_inventory_items(container: GridContainer, equipment_list: Array, index: int):
    for i in equipment_list.size():
        var equipment_item = equipment_list[i]
        var item_instance = item_scene.instantiate()
        item_instance.setup(equipment_item)
        container.add_child(item_instance)
        item_instance.highlight(i == index)
        if i == index:
            player_preview.add_equipment(equipment_item)
```

Allowing the Player to Unequip Hats

Finally, we want to allow the player to unequip hats from the menu. We will add an unequip function in both the player.gd and rogue.gd scripts.

In rogue.gd, add the following function:

```
func unequip():  
    for child in $Rig/Skeleton3D/Head.get_children():  
        child.queue_free()
```

In player.gd, add the following function:

```
func unequip():  
    for child in $PlayerSkin/Rogue/Rig/Skeleton3D/Head.get_children():  
        child.queue_free()
```

We will call these functions when the player presses the unequip button (mapped to the “run” action) in the inventory menu.

Here is the code snippet to handle the input in inventory.gd:

```
if Input.is_action_just_pressed("run"):  
    if tab_container.current_tab == 2:  
        player_preview.unequip()  
        player.unequip()
```

Fixing the Item Duplication Bug

There is a bug where items duplicate every time the menu is reopened. To fix this, we need to clear the items before recreating them. We will modify the reveal function in inventory.gd.

Here is the updated code snippet:

```
func reveal():  
    for child in tab_container.get_children():  
        for node in child.get_children():  
            node.queue_free()  
    show()  
    create_inventory_items(weapon_grid, player.weapons, player.weapon_index)  
    create_inventory_items(shield_grid, player.shields, player.shield_index)  
    create_inventory_items(style_grid, player.styles, player.style_index)  
    focus()
```

With these changes, our project is now complete. The player can equip and unequip items, and the menu displays the correct equipment with the appropriate colors.



In this learning article, we will recap and expand on the concepts covered in the video tutorial for creating an inventory system in Godot.

Recap of the Inventory System

In the video tutorial, we accomplished several key tasks to create an interactive inventory system in Godot:

- Utilized layout nodes and the layout system to design the inventory.
- Implemented a 3D preview using a subviewport to display equipment models.
- Added interactivity and controller input through GDScript.

Key Concepts Learned

Here are the key concepts and steps we covered:

1. **Layout Nodes:** We used layout nodes to organize and structure our inventory UI. This ensures that the inventory is responsive and adapts to different screen sizes.
2. **3D Preview:** By incorporating a subviewport, we provided players with a 3D preview of their equipment, enhancing the user experience.
3. **GDScript Logic:** We wrote GDScript to handle the logic for interactivity and controller input, making the inventory functional and user-friendly.

Integrating UI with the Game

A crucial aspect of game development is integrating the user interface with the actual game. In our project, we successfully merged the inventory UI with the base game, creating a cohesive experience.

Next Steps: Creating Levels

Now that we have a functional inventory system and a base game, the next step is to create levels to make the game more engaging. Here's what we plan to do:

- **Overworld and Castle Level:** We will design an overworld and a castle level for the player to explore.
- **Shaders for Enhanced Visuals:** To improve the visual appeal, we will add shaders for water, grass, and lava.

Conclusion

Creating an inventory system in Godot is a fundamental skill that enhances your game development capabilities. By following the steps outlined in this article, you will be well-equipped to design interactive and visually appealing user interfaces. Stay tuned for the next part of the series, where we will dive into creating levels and adding shaders to make your game even more immersive.

We hope you found this article helpful and look forward to seeing you in the next part of the series!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

global.gd

Found in folder: **/Global**

This code defines a set of dictionaries that store data for various game items, including weapons, shields, and style items. Each item has properties such as damage, defense, thumbnails, and scenes associated with it. The code also includes a function to pause and unpaue the game, which reveals or hides the inventory when paused.

extends Node

```
@onready var inventory = get_tree().get_first_node_in_group("Inventory")
var weapons: Dictionary = {
    'dagger':
    {
        'type': 'weapon',
        'damage': 1,
        'thumbnail': preload("res://graphics/thumbnails/dagger.png"),
        'scene': preload("res://scenes/weapons/dagger.tscn"),
        'animation': '1H_Melee_Attack_Stab',
        'range': 1.2,
        'audio': preload("res://audio/dagger_sound.wav")
    },
    'sword':
    {
        'type': 'weapon',
        'damage': 2,
        'thumbnail': preload("res://graphics/thumbnails/sword.png"),
        'scene': preload("res://scenes/weapons/sword.tscn"),
        'animation': '1H_Melee_Attack_Slice_Horizontal',
        'range': 1.5,
        'audio': preload("res://audio/sword_sound.wav")
    },
    'axe':
    {
        'type': 'weapon',
        'damage': 3,
        'thumbnail': preload("res://graphics/thumbnails/axe.png"),
        'scene': preload("res://scenes/weapons/axe.tscn"),
        'animation': '2H_Melee_Attack_Spin',
        'range': 1.3,
        'audio': preload("res://audio/axe_sound.wav")
    },
    'staff':
    {
        'type': 'weapon',
        'damage': 1,
        'thumbnail': preload("res://graphics/thumbnails/staff.png"),
```

```
'scene': preload("res://scenes/weapons/staff.tscn"),
'animation': '2H_Melee_Attack_Slice',
'range': 2.1,
'audio': preload("res://audio/staff_sound.wav")
},
}
var shields: Dictionary = {
'square': {
'type': 'shield',
'defense': 0.8,
'thumbnail': preload("res://graphics/thumbnails/square.png"),
'scene': preload("res://scenes/shields/square_shield.tscn")
},
'round': {
'type': 'shield',
'defense': 0.9,
'thumbnail': preload("res://graphics/thumbnails/round.png"),
'scene': preload("res://scenes/shields/round_shield.tscn")
},
'spike': {
'type': 'shield',
'defense': 0.6,
'thumbnail': preload("res://graphics/thumbnails/spike.png"),
'scene': preload("res://scenes/shields/spike_shield.tscn")
},
}
var style: Dictionary = {
'sunglasses': {
'type': 'style',
'scene': preload("res://scenes/style/sunglasses.tscn"),
'thumbnail': preload("res://graphics/thumbnails/sun_glasses.png")
},
'starglasses': {
'type': 'style',
'scene': preload("res://scenes/style/starglasses.tscn"),
'thumbnail': preload("res://graphics/thumbnails/star_glasses.png")
},
'duckhat': {
'type': 'style',
'scene': preload("res://scenes/style/duck_hat.tscn"),
'thumbnail': preload("res://graphics/thumbnails/duck.png")
},
'tophat': {
'type': 'style',
'scene': preload("res://scenes/style/tophat.tscn"),
'thumbnail': preload("res://graphics/thumbnails/top_hat.png")
},
}
}

func pause(value: bool = true):
if $PauseTimer.time_left == 0.0:
get_tree().paused = value
$PauseTimer.start()
if value:
```

```
inventory.reveal()
else:
    inventory.hide()
```

camera_controller.gd

Found in folder: **/scenes/characters**

This script controls the rotation of a 3D camera arm based on user input. It uses joystick or gamepad input to rotate the camera, with adjustable acceleration and limits on the rotation. The script also includes commented-out code for capturing mouse input and rotating the camera based on mouse movement.

```
extends SpringArm3D
```

```
@export var min_limit_x: float = -0.8
@export var max_limit_x: float = 0.2
@export var horizontal_acceleration: float = 2
@export var vertical_acceleration: float = 1
@export var mouse_acceleration: float = 0.005

func _process(delta: float) -> void:
    var joy_dir = Input.get_vector("pan_left", "pan_right", "pan_up", "pan_down")
    var joy_dir_accelerated = joy_dir * delta * Vector2(horizontal_acceleration, vertical_acceleration)
    rotate_from_vector(joy_dir_accelerated)

#func _ready() -> void:
#    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
#
#func _input(event: InputEvent) -> void:
#    #if event is InputEventMouseMotion:
#        #var mouse_dir_accelerated = event.relative * mouse_acceleration
#        #rotate_from_vector(mouse_dir_accelerated)
#    #if Input.is_action_just_pressed("ui_cancel"):
#        #get_tree().quit()

func rotate_from_vector(v: Vector2):
    if v.length() == 0 : return
    rotation.y -= v.x
    rotation.x -= v.y
    rotation.x = clamp(rotation.x, min_limit_x, max_limit_x)
```

character.gd

Found in folder: **/scenes/characters**

This script defines the behavior of a character in a 3D game, including movement, jumping, attacking, defending, and health management. It also handles animation states, equipment management, and collision detection. The script uses a finite state machine to manage the character's movement and animation states.

```
class_name Character
extends CharacterBody3D

@export var base_speed: float = 4
@export var run_speed: float = 6
@export var defend_speed: float = 2
var movement_input: Vector2

@export var jump_height: float = 2.25
@export var jump_time_to_peak: float = 0.4
@export var jump_time_to_descent: float = 0.3

@onready var jump_velocity: float = ((2.0 * jump_height) / jump_time_to_peak) * -1.0
@onready var jump_gravity: float = ((-2.0 * jump_height) / (jump_time_to_peak * jump_time_to_peak)) * -1.0
@onready var fall_gravity: float = ((-2.0 * jump_height) / (jump_time_to_descent * jump_time_to_descent)) * -1.0

@onready var move_state_machine = $AnimationTree.get("parameters/MoveStateMachine/playback") as AnimationNodeStateMachinePlayback
@onready var attack_animation = $AnimationTree.get_tree_root().get_node('AttackAnimation') as AnimationNodeAnimation

var skin: Node3D
var attacking: bool = false
var defending: bool = false:
    set(value):
        if not defending and value:
            defend_toggle(true)
        if defending and not value:
            defend_toggle(false)
        defending = value
var current_weapon: Node3D
var current_shield: Node3D
var squash_and_stretch: float = 1.0:
    set(value):
        squash_and_stretch = value
        var negative = 1.0 + (1.0 - squash_and_stretch)
        skin.scale = Vector3(negative, squash_and_stretch, negative)
var health: int = 3:
    set(value):
        if "hud" in self:
            get_tree().get_first_node_in_group("HUD").set_health(value)
        if value <= 0:
            get_tree().quit()
        health = value

func defend_toggle(forward: bool):
    var tween = create_tween()
    tween.tween_method(_defend_change, 1.0 - float(forward), float(forward), 0.25)

func _defend_change(value):
    $AnimationTree.set("parameters/DefendBlend/blend_amount", value)
```

```
func apply_gravity(gravity, delta):
    velocity.y -= gravity * delta

func set_move_state(state_name: String):
    move_state_machine.travel(state_name)

func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
        attack_animation.animation = data['animation']
        current_weapon = item_scene
        current_weapon.set_sound(data['audio'])
    if data['type'] == 'shield':
        item_scene.defense = data['defense']
        current_shield = item_scene

func _on_animation_tree_animation_finished(anim_name: StringName) -> void:
    if 'Attack' in anim_name:
        attacking = false

func attack_logic():
    if attacking:
        var collider = current_weapon.get_collider()
        if collider and collider != self and "hit" in collider:
            collider.hit(current_weapon)

func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
        if current_shield and defending:
            health -= attacking_weapon.damage * current_shield.defense
            current_shield.flash()
            current_shield.play_audio()
        else:
            health -= attacking_weapon.damage
            $HitSound.play()
            $Timers/HitTimer.start()
            do_squash_and_stretch(1.2, 0.2)

    if health <= 0:
        death_logic()

func death_logic():
    pass
```

```
func do_squash_and_stretch(value: float, duration: float):
    var tween = create_tween()
    tween.tween_property(self, "squash_and_stretch", value, duration)
    tween.tween_property(self, "squash_and_stretch", 1.0, duration * 1.8).set_ease(Tween.EASE_OUT)
```

player.gd

Found in folder: **/scenes/characters**

This script controls the player character's movement, actions, and equipment. It handles input for movement, jumping, attacking, and switching between weapons and shields. The script also updates the player's animation state and applies gravity and movement physics.

extends Character

```
@export var acceleration: float = 8
@export var deceleration: float = 4
@onready var camera = $CameraController/Camera3D
@onready var hud = get_tree().get_first_node_in_group("HUD")

var weapons = [Global.weapons['sword'], Global.weapons['dagger'], Global.weapons['staff']]
var weapon_index: int
var shields = [Global.shields['round'], Global.shields['square'], Global.shields['spike']]
var shield_index: int
var styles = [Global.style['duckhat']]
var style_index: int = -1

func _ready() -> void:
    skin = $PlayerSkin
    equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
    #equip(styles[style_index], $PlayerSkin/Rogue/Rig/Skeleton3D/Head)
    hud.setup(health)

func _physics_process(delta: float) -> void:
    move_logic(delta)
    jump_logic(delta)
    ability_logic()
    move_and_slide()
    attack_logic()

func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("switch_weapon"):
        weapon_index = posmod(weapon_index + 1, weapons.size())
        equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    if Input.is_action_just_pressed("switch_shield"):
        shield_index = posmod(shield_index + 1, shields.size())
```



```
equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
if Input.is_action_just_pressed("ui_focus_next"):
    health += 1
if Input.is_action_just_pressed("ui_cancel"):
    health -= 1
if Input.is_action_just_pressed("menu"):
    Global.pause()

func move_logic(delta):
    movement_input = Input.get_vector("left", "right", "forward", "backward").rotated(-camera.global_rotation.y)
    var vel_2d = Vector2(velocity.x, velocity.z)

    if movement_input != Vector2.ZERO:
        var speed = run_speed if Input.is_action_pressed("run") else base_speed
        speed = defend_speed if defending else speed
        vel_2d += movement_input * speed * delta * acceleration
        vel_2d = vel_2d.limit_length(speed)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
        var target_angle = -movement_input.angle() + PI / 2
        skin.rotation.y = rotate_toward(skin.rotation.y, target_angle, delta * 6)
        set_move_state('Running_A')
    else:
        vel_2d = vel_2d.move_toward(Vector2.ZERO, base_speed * delta * deceleration)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
        set_move_state('Idle')

func jump_logic(delta):
    if is_on_floor():
        if Input.is_action_just_pressed("jump"):
            velocity.y = -jump_velocity
    else:
        set_move_state('Jump_Idle')
    var gravity = jump_gravity if velocity.y > 0.0 else fall_gravity
    apply_gravity(gravity, delta)

func ability_logic():
    defending = Input.is_action_pressed("defend")
    if Input.is_action_just_pressed("attack"):
        if not attacking:
            $AnimationTree.set("parameters/AttackOneShot/request", AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
            attacking = true
            current_weapon.play_audio()

func death_logic():
    get_tree().quit()
```

```
func unequip():  
    for child in $PlayerSkin/Rogue/Rig/Skeleton3D/Head.get_children():  
        child.queue_free()
```

skeleton.gd

Found in folder: **/scenes/characters**

This code controls an enemy character that moves towards the player and attacks when within a certain range. The enemy's movement and attack logic are updated every physics frame, and it also handles death animations and collision disabling when its health reaches zero. The enemy's appearance and equipment are randomized when it is initialized.

```
extends Character
```

```
@onready var player = get_tree().get_first_node_in_group('Player')  
@export var notice_radius: float = 30  
@export var speed: float = 3  
var attack_radius: float = 3  
var rng = RandomNumberGenerator.new()
```

```
func _ready() -> void:  
    for child in $Skins.get_children():  
        child.hide()  
    skin = $Skins.get_children().pick_random()  
    skin.show()  
    $AnimationTree.anim_player = "../Skins/" + skin.name + '/AnimationPlayer'  
    var random_weapon = Global.weapons[Global.weapons.keys().pick_random()]  
    equip(random_weapon, skin.get_node('Rig/Skeleton3D/RightHand'))  
    attack_radius = current_weapon.radius  
    health = 3
```

```
func _physics_process(delta: float) -> void:  
    if health > 0:  
        move_to_player(delta)  
        if not is_on_floor():  
            apply_gravity(fall_gravity, delta)  
        else:  
            velocity.y = 0  
        move_and_slide()  
        attack_logic()
```

```
func move_to_player(delta):  
    if position.distance_to(player.position) < notice_radius:  
        var target_dir = (player.position - position).normalized()  
        var target_vec2 = Vector2(target_dir.x, target_dir.z)  
        var target_angle = -target_vec2.angle() + PI/2  
        rotation.y = rotate_toward(rotation.y, target_angle, delta * 6)  
    if position.distance_to(player.position) > attack_radius:  
        velocity = Vector3(target_vec2.x, 0, target_vec2.y) * speed
```



```
        set_move_state('Running_A')
    else:
        velocity = Vector3.ZERO
        set_move_state('Idle')

func _on_attack_timer_timeout() -> void:
    $Timers/AttackTimer.wait_time = rng.randf_range(2.0, 3.5)
    if position.distance_to(player.position) < attack_radius:
        $AnimationTree.set('parameters/AttackOneShot/request', AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
        attacking = true
        current_weapon.play_audio()

func death_logic():
    $CollisionShape3D.disabled = true
    $Timers/AttackTimer.stop()
    var tween = create_tween()
    tween.tween_method(_death_change, 0.0, 1.0, 0.25)

func _death_change(value):
    $AnimationTree.set("parameters/DeathBlend/blend_amount", value)
```

shield.gd

Found in folder: **/scenes/shields**

This script extends the Node3D class and represents a shield equipment with a defense value. It has a flash function that creates a tween to temporarily change the alpha value of a material overlay, causing a flashing effect. The script also has a play_audio function that plays a sound effect when called.

```
extends Node3D

var defense: float
const equipment_type = 'Shield'

func flash():
    var tween = create_tween()
    tween.tween_method(_flash, 0.0, 1.0, 0.2)
    tween.tween_method(_flash, 1.0, 0.0, 0.4)

func _flash(value):
    get_child(0).get_child(0).material_overlay.set_shader_parameter('alpha', value)

func play_audio():
    $HitSound.play()
```

hud.gd

Found in folder: **/scenes/ui**

This code creates a control that displays a set of hearts, where the number of hearts is determined by the `setup` function. The `setup` function instantiates a specified number of heart scenes and adds them as children to the `$Hearts` node. The `set_health` function clears any existing hearts and then calls `setup` to display the new number of hearts.

```
extends Control

var heart_scene = preload("res://scenes/ui/heart.tscn")

func setup(hearts: int):
    for heart in hearts:
        var heart_instance = heart_scene.instantiate()
        $Hearts.add_child(heart_instance)

func set_health(health: int):
    for child in $Hearts.get_children():
        child.queue_free()
    setup(health)
```

inventory.gd

Found in folder: **/scenes/ui**

This code manages the player's inventory and equipment selection. It populates grid containers with equipment items, allows the player to navigate and select equipment, and updates the player preview accordingly. The code also handles input events, such as panning the player preview and switching between equipment tabs.

```
extends Control

@onready var player_preview = get_tree().get_first_node_in_group("PlayerPreview")
@onready var player = get_tree().get_first_node_in_group("Player")
@onready var item_scene = preload("res://scenes/ui/item.tscn")
@export var pan_speed:float = 3

@onready var weapon_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Weapons
@onready var shield_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Shields
@onready var style_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Style
@onready var tab_container = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer
const border_colors = [Color.LIME_GREEN, Color.FIREBRICK, Color.GOLDENROD]

func reveal():
    for child in tab_container.get_children():
        for node in child.get_children():
            node.queue_free()
    show()
```

```
create_inventory_items(weapon_grid, player.weapons, player.weapon_index)
create_inventory_items(shield_grid, player.shields, player.shield_index)
create_inventory_items(style_grid, player.styles, player.style_index)
focus()

func focus():
    await get_tree().create_timer(0.01).timeout
    if tab_container.get_child(tab_container.current_tab).get_child_count():
        tab_container.get_child(tab_container.current_tab).get_child(0).grab_focus()

func _process(delta: float) -> void:
    var pan_dir = Input.get_axis("pan_left", "pan_right")
    player_preview.rotate_y(pan_dir * delta * pan_speed)

func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("menu"):
        Global.pause(false)
    var menu_dir = int(Input.get_axis("switch_shield", "switch_weapon"))
    if menu_dir:
        tab_container.current_tab = posmod(tab_container.current_tab + menu_dir, tab_container.get_child_count())
        focus()
        $HBoxContainer/ItemPanelContainer/Control/Label.text = tab_container.get_child(tab_container.current_tab).name
        var color = border_colors[tab_container.current_tab]
        $HBoxContainer/ItemPanelContainer/MarginContainer/Panel.get_theme_stylebox("panel").border_color = color
        $HBoxContainer/ItemPanelContainer/Control/Label.add_theme_color_override("font_color", color)
    if Input.is_action_just_pressed("run"):
        if tab_container.current_tab == 2:
            player_preview.unequip()
            player.unequip()

func create_inventory_items(container: GridContainer, equipment_list: Array, index: int):
    for i in equipment_list.size():
        var equipment_item = equipment_list[i]
        var item_instance = item_scene.instantiate()
        item_instance.setup(equipment_item)
        container.add_child(item_instance)
        item_instance.highlight(i == index)
        if i == index:
            player_preview.add_equipment(equipment_item)
```

item.gd

Found in folder: **/scenes/ui**

This script extends the Button class and is used to display and manage equipment items. It sets up

the button's appearance and behavior based on the provided equipment data, and handles user interactions such as highlighting and equipping the item. When pressed, the button updates the player's equipment and preview, and highlights itself as the currently equipped item.

```
extends Button
```

```
@onready var player_preview = get_tree().get_first_node_in_group("PlayerPreview")
@onready var player = get_tree().get_first_node_in_group("Player")
var equipment_data
```

```
func _ready() -> void:
    size_flags_horizontal = Control.SIZE_EXPAND | Control.SIZE_FILL
```

```
func setup(data):
    $TextureRect.texture = data['thumbnail']
    equipment_data = data
```

```
func _on_resized() -> void:
    custom_minimum_size.y = get_rect().size.x
```

```
func highlight(value: bool):
    if value:
        $EquippedPanel.show()
    else:
        $EquippedPanel.hide()
```

```
func _on_pressed() -> void:
    # update the real player
    var slot = {'weapon': 'RightHand', 'shield': 'LeftHand', 'style': 'Head'}[equipment_data['type']]
    player.equip(equipment_data, player.get_node('PlayerSkin/Rogue/Rig/Skeleton3D/' + slot))

    #update the preview
    player_preview.add_equipment(equipment_data)
    for item in get_parent().get_children():
        item.highlight(false)
    highlight(true)
```

rogue.gd

Found in folder: **/scenes/ui**

This code adds equipment to a 3D character model by instantiating a scene and attaching it to a specific slot on the character's rig. The slot is determined by the type of equipment being added, such as a weapon, shield, or style item. The code also provides a function to unequip items from the character's head slot, removing any child nodes from that location.

```
extends Node3D
```

```
func add_equipment(data):
    var slot = {
        'weapon': $Rig/Skeleton3D/RightHand,
        'shield': $Rig/Skeleton3D/LeftHand,
        'style': $Rig/Skeleton3D/Head
    }[data['type']]
    for child in slot.get_children():
        child.queue_free()
    var item_instance = data['scene'].instantiate()
    slot.add_child(item_instance)

func unequip():
    for child in $Rig/Skeleton3D/Head.get_children():
        child.queue_free()
```

weapon.gd

Found in folder: **/scenes/weapons**

This script defines a 3D node that represents a weapon in the game. It stores properties such as animation, damage, and radius, which are set up through the `setup` function. The script also provides methods to get the collider, set and play audio for the weapon's attack sound.

```
extends Node3D
```

```
const equipment_type = 'Weapon'
var animation: String
var damage: int
var parent
var radius: float
```

```
func setup(weapon_animation, weapon_damage, weapon_radius, weapon_parent):
    animation = weapon_animation
    damage = weapon_damage
    parent = weapon_parent
    radius = weapon_radius
```

```
func get_collider():
    return $RayCast3D.get_collider()
```

```
func set_sound(audio):
    $AttackSound.stream = audio
```

```
func play_audio():
    $AttackSound.play()
```



flash.gdshader

Found in folder: **/shaders**

This shader sets the color and transparency of an object in 3D space. It uses two variables, “color” and “alpha”, which can be adjusted to change the object’s appearance. The shader applies these values to the object’s material, with “color” controlling the object’s hue and “alpha” controlling its opacity.

```
shader_type spatial;

uniform float alpha: hint_range(0.0, 1.0) = 0.0;
uniform vec3 color: source_color = vec3(1.0);

void fragment() {
    ALBEDO = color;
    ALPHA = alpha;
}
```